Starter PRD: Session Seat Desk

Completion Theater Hackathon, Agile Testing Days 2026

Build a small application that reserves seats in two fictional workshops. Your team has 60 minutes to build, followed by verification and an independent cross-audit. Any language, framework or AI coding agent is welcome. The product must run on a participant laptop without a paid deployment, a private service or a judge's AI account.

The user and the problem

An event helper needs to see remaining seats, reserve a place for an attendee and cancel a reservation. A successful-looking screen is not enough: the stored state must agree with the response, survive a restart and never oversell a session.

Use fictional aliases only. There is no login, payment, email delivery, public hosting or real conference registration in this exercise. Keep your app and its test data in a fresh, disposable team workspace.

The smallest complete product

Provide a basic user interface and a local JSON HTTP API. The interface may be a simple HTML page. The API lets another team independently inspect behavior. Store state durably, for example in SQLite or a local file. Document the state location. An in-memory array or browser-only state does not meet the persistence requirement. A local process must accept a configurable port and isolated fixture state path, either directly or through a documented adapter. The optional Python scaffold supports PORT and DATA_DIR. Labels, keyboard access and visible error messages belong in the basic UI, not optional creativity work.

Seed exactly these sessions on an explicit fresh reset. Restart must not reset them.

Session ID	Title	Capacity	Initial active reservations
CT-101	Catching Ghost Databases	2	0
CT-102	Tests That Can Fail	3	0

A reservation contains a generated stable ID, session ID, attendee alias, caller-supplied request key and status (active or cancelled). Remaining seats equal capacity minus active reservations. Reject blank or whitespace-only aliases. Trim aliases before comparing repeat requests; comparisons are otherwise case-sensitive. The request key must be a non-empty string.

Required acceptance criteria

Each criterion is worth 5 of the 40 working-app points. PASS requires observed execution against the frozen submission. FAIL or NOT RUN earns 0 for that criterion. Partial implementations can still earn other criteria.

ID	Requirement and observable acceptance
AC-01	Clean start and UI. Another team follows the README, starts the app from the submitted source, opens the UI and sees the two seeded sessions. App startup does not require the builder's account or a hidden service.
AC-02	Session readback. GET /sessions returns both IDs, capacities, active counts and remaining seats. On fresh reset, counts are 0 and remaining seats are 2 and 3. The UI agrees with this response.
AC-03	Reservation. A valid request for CT-101 with alias Ada and key K1 returns success with a stable reservation ID and active status. GET /reservations/{id} returns the same record. Remaining CT-101 seats become 1 and the UI reflects that state.

ID	Requirement and observable acceptance
AC-04	Capacity. After two active CT-101 reservations, a third distinct request returns a non-success response with a clear capacity error and creates no reservation. A fresh one-seat state with simultaneous distinct reservation attempts must produce exactly one success and an active count of 2; all other attempts must reject without state changes. The shared helper repeats ten bounded batches of ten attempts. This probes races; it does not prove absence of all concurrency defects.
AC-05	Idempotency. Repeating the same key, same session and same trimmed alias returns the original reservation ID and current status without consuming another seat. Reusing that key with a different session or alias returns a conflict with no state change. Request keys are globally unique, not just unique within a session.
AC-06	Cancellation. Cancelling an active reservation releases exactly one seat. Repeating cancellation reports cancelled status and leaves the count unchanged. Replaying its original reservation request after cancellation returns that same cancelled reservation; it must not reactivate it. A new request key is needed for a new reservation.
AC-07	Durable state. After creating one active and one cancelled reservation, stop the process, start it again without reset, and use fresh HTTP requests plus the UI to read the same IDs, statuses and seat counts. Refreshing a browser alone is not a restart.
AC-08	Invalid input. Unknown session, blank alias, missing request key, malformed JSON and cancelling an unknown reservation produce non-success responses with clear errors. None may change reservation counts or remaining seats. The process stays available afterward.

Shared API contract

Rules version 1.1, updated after the 17 October dry run. Paths are fixed so another team can test quickly. Use the shared JSON examples in the public repository API contract, or supply a tested adapter for your response field names before the build ends. The optional verifier accepts both a bare session array and the documented wrapper through that adapter. Return valid JSON and appropriate HTTP status codes; any documented 2xx success and 4xx rejection is acceptable. A 200 response containing only an error message is not a rejection.

Method and path	Input	Minimum result
GET /sessions	none	session IDs, titles, capacities, active counts, remaining seats
POST /reservations	sessionId, attendeeAlias, requestKey	stable reservation ID, session, alias, key, status
GET /reservations/{id}	ID from a successful response	stored reservation or 4xx if unknown
POST /reservations/{id}/cancel	empty JSON object	current cancelled record or 4xx if unknown

Example request: {"sessionId":"CT-101","attendeeAlias":"Ada","requestKey":"K1"}.

Provide a documented local reset command for fixture state. It must target only this disposable workspace. Never reset automatically at startup. A reset HTTP endpoint is optional; do not expose one publicly. Auditors can stop and reset their own copy between independent scenarios, but never between the stop and restart steps of AC-07.

Invariants to investigate

- Active count never exceeds capacity and remaining seats never go below 0.

- Replays, rejected input and repeated cancellation do not create extra state changes.

- A successful response agrees with independent stored-state readback.

- Restart preserves records and counts.

- Test failure reaches the process exit status; skipped or absent tests are visible.

Scope and optional ideas

Finish the eight criteria before adding waitlists, search, accessibility improvements or evidence views. Authentication, real personal data, payments, live messaging and production deployment are outside this challenge. Do not invent AI features for the running app: AI is your build assistant, not a required product dependency.

What to hand over

Use one fork and submission branch per team. At 21:55 CET commit and push the final source/tests, record the full 40-character commit SHA and declared app directory, and stop pushing to that branch. By 22:00 open a PR to organizer main with the completed submission fields and redacted evidence location. The exact commit, not the moving PR branch or merge preview, is judged. Follow docs/08-Submission-Workflow.md for commands, retained snapshots and the facilitator-assisted offline fallback.

Include README run/reset/test commands, runtime version, dependency lockfiles where supported, state location, API field mapping, test source and known gaps inside the declared app directory. Provide no credentials, AI tokens, private .env files, unrelated history or real attendee records. Use .env.example for configuration names only.

Keep evidence and audit reports outside the app directory and frozen source archive. Deliver them through PR attachments/comments or the facilitator-recorded transfer path, not new commits on the frozen branch. Bind each evidence record to the full commit and snapshot sourceId. If your source changes before freeze, rerun affected checks. Work after freeze belongs in a separately named copy and does not replace the judged baseline.

Builder verification checklist

Team: _____ Verifier: _____

PR URL / number: _____ App directory: _____

Frozen full commit: _____ Snapshot sourceId: _____

Archive SHA-256 (if transferred): _____

Verification window: 21:40–22:00. Fixes are allowed until 21:55. Freeze source and tests at 21:55; package evidence during the final five minutes. Do not add features during this round.

Evidence rules

Record a result for every row: PASS, FAIL or NOT RUN. Add the evidence class REPORTED, STATIC or EXECUTED. Only an EXECUTED PASS earns working-app points. A static code inspection or an agent's claim is useful context, but cannot establish runtime success. A deliberate rejection is a PASS when rejection and unchanged state are the expected behavior.

Run evidence against a source content hash before and after the check. Earlier receipts count only when both hashes equal the final frozen source identity. If the source changes, rerun affected checks. The shared verifier records this binding; no claim can be rescued by a later hash alone.

For each executed check capture the criterion ID, source identity, command or UI steps, expected result, actual result, status/exit code, timestamp and artifact path. Screenshots help with UI behavior; keep raw responses and test logs too. Redact secrets before sharing. Do not use coverage percentages as a verdict that the app works.

Functional verification

Use a fresh fixture reset at the start of each scenario. Do not reset inside a scenario unless explicitly instructed. Record NOT RUN honestly when time expires.

Check	Steps and oracle	Result / class / evidence path
AC-01	Follow README from a fresh source copy, install documented dependencies, start app and open UI. No builder account or hidden service.	_____
AC-02	Reset fixture. GET /sessions and inspect UI. CT-101: capacity 2, active 0, remaining 2. CT-102: 3, 0, 3.	_____
AC-03	Reset fixture. Reserve CT-101 / Ada / K1. Save returned ID. Read it through GET /reservations/{id}; check remaining 1 and UI agreement.	_____
AC-04a	Reset. Reserve CT-101 with K1 and K2. Try K3. Expect two successes, a 4xx capacity rejection, active 2, remaining 0.	_____
AC-04b	Reset. Create one CT-101 booking. Send K2 and K3 concurrently. Expect exactly one new success and one rejection; active 2, remaining 0. Use ten bounded batches when using the shared helper. Record concurrency and limits; passing a bounded race does not prove atomicity. AC-04 requires both 04a and 04b.	_____
AC-05	Reset. Send Ada / CT-101 / K1 twice, including a whitespace-trimmed alias replay. Same ID, active 1. Reuse K1 with a different alias and then CT-102: both conflicts, no count change.	_____
AC-06	Reset fixture. Create CT-101 / Ada / K1 and save its ID. Cancel it. Repeat cancellation. Remaining becomes 2 and stays 2. Replay original K1 create request: same cancelled ID, still remaining 2.	_____

Check	Steps and oracle	Result / class / evidence path
AC-07	Reset. Make K1 and K2 in CT-101; cancel K1. Capture IDs, statuses, active 1, remaining 1. Stop process. Restart without reset. Fresh HTTP and UI readback must match. Record state path and PID/process restart.	_____
AC-08	Reset. Try each invalid-input case in PRD. Capture every rejection and before/after counts for both sessions. GET /sessions must still work.	_____

Five completion theater checks

Pattern	Investigation	Result / evidence
Ghost Database	Show actual persisted records and state path. Restart and query again. Confirm the app reads the same store it writes.	_____
Metric Theater	Trace any test count or coverage claim to a fresh tool output. If no coverage tool ran, say coverage was not measured.	_____
Test Theater	Read one important assertion. In a disposable copy, deliberately break the behavior or expected result and rerun that test. It must fail. Restore the copy. No code mutation in the frozen baseline.	_____
False Positives	Inspect raw stdout/stderr, exit status, skipped tests and start failures. Trigger a known failing check and confirm non-zero exit. A green wrapper cannot hide failure.	_____
Uncommitted Illusion	Extract the exact handed-over archive or check out the recorded commit in a fresh disposable directory. Confirm the required files and tests are present. Local files omitted from the artifact do not count.	_____

Evidence score preparation

The judge awards six binary items, 5 points each: exact source binding; reproducible run/reset/test instructions; raw executed results including failure/skip counts and exit codes; durable-state readback; a failing negative control for a meaningful test; and an honest claim-to-evidence manifest including gaps. These assess the evidence, separately from whether the app meets each acceptance criterion.

Final handoff

[ ] Source/tests committed and pushed at 21:55; full commit and app directory recorded.

[ ] PR to organizer main submitted by 22:00; facilitator records receipt and retains the exact snapshot.

[ ] Snapshot sourceId and archive SHA-256 recorded; the retained archive opens.

[ ] README covers runtime, install, start, reset, test, state path and API mapping.

[ ] Evidence manifest and redacted receipts are delivered outside the frozen source through the PR or recorded transfer path. They point to readable logs and UI evidence, not agent summaries alone.

[ ] Known failures and unexecuted checks are visible.

[ ] Private configuration, credentials and unrelated files are excluded.

[ ] Auditor has a working transfer path and a disposable directory of their own.

[ ] Any post-freeze repair is separate and cannot overwrite the judged baseline.

Cross-audit checklist and findings sheet

Auditing team: _____ Target team: _____ Facilitator: _____

Target source identity: _____

Audit window: 22:05–22:40. Use the facilitator-recorded PR, full frozen commit, app path and retained snapshot, not the current branch tip. Inspect that frozen artifact in a disposable copy. Keep your audit receipts outside the target source and report them with your auditing team ID and target identity. The builder can explain commands but does not operate the auditor's checks. Agree to the review of this fixture app only. Never inspect unrelated directories, accounts, production systems or personal files.

A 35-minute investigation

Minutes	Action	Receipt
0–5	Check source identity, README, dependencies and state path. Start from the supplied artifact. If blocked, spend at most 5 minutes before notifying a facilitator.	_____
5–15	Independently run a happy path and inspect persisted state. Compare responses, UI and restart behavior.	_____
15–25	Challenge capacity, concurrent reservations, request replay, cancelled-key replay or malformed input. Pick high-risk checks before cosmetic issues.	_____
25–30	Challenge one test with a deliberate failure in a copy, or trace a claimed metric to actual output. Record what the evidence can and cannot establish.	_____
30–35	Write at most two strongest findings/investigations. Share reproduction steps with the builder and facilitator. Record disagreements.	_____

Pattern prompts

- Ghost Database: did a record survive a real process restart, or just a refresh?

- Metric Theater: where did the claimed number come from, and what does it mean?

- Test Theater: can an important check actually fail when the behavior is wrong?

- False Positives: did the process fail, skip work or swallow an error while reporting success?

- Uncommitted Illusion: does the frozen artifact contain the implementation and tests being demonstrated?

Do not force every defect into a named pattern. An ordinary functional defect can be important. A suspected theater pattern becomes a finding only when evidence supports it.

Finding or bounded clean investigation

Copy this form for each of your two strongest investigations. A clean result can earn cross-audit credit when you demonstrate a serious challenge, its oracle and its bounded conclusion. This avoids rewarding invented bugs or penalizing a team assigned a sound target.

Investigation ID: _____ Target criterion / claim: _____

Expected behavior / oracle: _____

Setup and exact reproduction steps: _____

_____

Actual result, error and state readback: _____

Evidence files, timestamp and source identity: _____

Classification: [ ] Confirmed mismatch [ ] Not reproduced [ ] Bounded clean result [ ] Blocked

Pattern (if supported): _____

Impact: [ ] Core behavior [ ] Evidence integrity [ ] Usability [ ] Other

Builder response / disagreement: _____

Facilitator confirmation: _____ Follow-up: _____

Fairness and scope

No edits to the frozen source. No deliberate planting of defects in the target baseline. Temporary negative controls belong only in a clearly marked disposable copy. Duplicate symptoms with one root cause are one finding. Suggestions are not defects. Unreproduced suspicions must remain labelled. Facilitators adjudicate contested results against the same PRD and source identity.

Cross-audit points belong to the auditing team. The target team's working-app score follows its acceptance results. Do not subtract extra points merely because a peer found a defect. Builders earn evidence credit for an honest FAIL, not for hiding it.

Judge scoring sheet

Team: _____ Judge: _____ Source identity: _____

Published weights: working app 40, verification evidence 30, cross-audit catches/challenges 20, creativity 10. Total 100. Rewards remain open. Rubric version 1.1. Announce this same rubric to all teams before the build; changes after that need an equal, recorded ruling.

Every row scores either 0 or 5. Mark the observation PASS, FAIL or NOT RUN, with an evidence reference. Only EXECUTED PASS supports functional points. Missing evidence earns 0, not an inferred pass. A test that correctly rejects invalid input is a PASS.

A. Working app: 40 points

ID	Criterion	0 or 5	Evidence
A1 / AC-01	Reproducible start and usable basic UI	____	_____
A2 / AC-02	Seeded sessions, API counts and UI agreement	____	_____
A3 / AC-03	Valid reservation and independent record readback	____	_____
A4 / AC-04	Capacity holds sequentially AND under concurrent requests	____	_____
A5 / AC-05	Same-key replay and conflicting-key rejection	____	_____
A6 / AC-06	Cancellation, repeated cancellation and cancelled-key replay	____	_____
A7 / AC-07	Active and cancelled state survives process restart	____	_____
A8 / AC-08	All five invalid-input cases reject without state changes	____	_____

Working-app subtotal (0–40): _____

B. Verification evidence: 30 points

ID	Criterion	0 or 5	Evidence
B1	Frozen source hash/checksum and each receipt bound to it	____	_____
B2	Run/reset/test instructions independently reproduced	____	_____
B3	Fresh raw test output shows exit codes, failures and skipped work	____	_____
B4	Before/after state evidence includes a real process restart	____	_____
B5	A meaningful test fails under a deliberate negative control in a copy	____	_____
B6	Manifest distinguishes claims, executed results, FAIL and NOT RUN	____	_____

Evidence subtotal (0–30): _____

B3 can earn 5 even when tests fail, if the output is complete and honestly reported. B4 requires readable state evidence and a recorded observation; matching state earns AC-07 separately. B5 requires an important behavioral assertion, not just a syntax error. A known limitation must be disclosed, not fabricated to satisfy B6.

C. Cross-audit: 20 points

Target team: _____ Investigation references: _____

ID	Criterion	0 or 5	Evidence
C1	Auditor independently starts the exact frozen artifact	____	_____
C2	Auditor executes a high-risk functional challenge with an explicit oracle	____	_____
C3	Auditor challenges an evidence claim, test sensitivity or persistence independently	____	_____
C4	Report contains a reproducible mismatch OR a bounded clean result, plus limits and builder response	____	_____

Cross-audit subtotal (0–20): _____

Record the auditing team and target separately. Do not add points for an audit performed ON this team to its own score. A team must perform an audit to earn C points. An AI-only rehearsal is labelled rehearsal, not an independent human-team qualification.

No points per bug, no bonus for dramatic severity labels, no requirement to invent a finding. At most two investigations are considered. A blocked cold start can earn C4 with a reproducible, adjudicated report, but does not earn C1.

D. Creativity: 10 points

ID	Criterion	0 or 5	Evidence
D1	One optional improvement solves a concrete user problem and is demonstrated	____	_____
D2	One additional verification technique is demonstrated beyond the supplied checks, with a clear question and result	____	_____

Creativity subtotal (0–10): _____

Both can earn credit in a partial app; an unsupported feature promise cannot. Repeating the mandatory capacity check is not an additional technique. Novelty is judged relative to this starter, not to the wider industry.

Final decision

A ____ + B ____ + C ____ + D ____ = TOTAL ____ / 100

Judge and second facilitator confirm arithmetic: _____

Tie-break order: higher B, then higher C, then higher A. If still tied, share the position. Do not improvise a new criterion after the build. A new disputed rule applies to everyone or is left unscored; facilitators record the decision.

No penalty for selecting a cheaper tool, not using AQE, avoiding deployment, acknowledging a failure or receiving no defects from an auditor. Unsafe activity is stopped and the affected evidence is invalidated; unrelated earned points remain unless event organizers decide otherwise.

Master scoreboard

Event: 18 November 2026 Judge: _____

Team	Working /40	Evidence /30	Audit /20	Creativity /10	Total /100	Tie order / notes
A	____	____	____	____	____	_____
B	____	____	____	____	____	_____
C	____	____	____	____	____	_____
D	____	____	____	____	____	_____
E	____	____	____	____	____	_____
F	____	____	____	____	____	_____
G	____	____	____	____	____	_____
H	____	____	____	____	____	_____

Score owner: _____ Arithmetic checked by: _____

Outstanding disputes: _____

Rewards: OPEN. Do not announce products, subscriptions, values or sponsorship commitments.

Completion Theater: field card

The Completion Theater five-pattern field guide, adapted for tonight's investigation.

Pattern	The claim	The check
Ghost Database	'The data is saved.'	Read the real record, restart the process, query again and check the app's state path.
Metric Theater	'Excellent coverage. All tests pass.'	Locate the fresh tool output. Check scope, denominator, skipped work and exit status. No metric is required if it was not measured.
Test Theater	'The test proves it works.'	Read the assertion. Deliberately break the relevant behavior or expected result in a disposable copy. The check must fail.
False Positives	'Success.'	Inspect the raw response and exit code. Trigger a known failure and confirm the wrapper propagates it.
Uncommitted Illusion	'The implementation is finished.'	Start from the exact delivered commit or checksummed archive. Check that the demonstrated source and tests are present.

A useful evidence receipt

Claim / criterion: _____

Source identity: _____ Time: _____

Setup and command / UI steps: _____

Expected: _____ Observed: _____

Result: [ ] PASS [ ] FAIL [ ] NOT RUN

Evidence class: [ ] REPORTED [ ] STATIC [ ] EXECUTED

Raw log / state / screenshot path: _____

What this does not establish: _____

Three reminders

An agent's conclusion is a claim to investigate. An executed check has a scope. A clean result means 'not observed within these checks', not 'no defects exist'.

A bug is not automatically completion theater. Name the pattern only when the evidence supports that interpretation. Discuss the artifact and the claim, not the competence of its authors.